

"Resumen" Redes y Comunicaciones

Clase 1

¿Qué es una red de computadoras/ordenadores?

- Análisis del punto de vista sistémico:
 - Red de computadoras: Grupo de computadoras/dispositivos interconectados
 - Objetivo: Compartir recursos (dispositivos, información, servicios).
 - El conjunto de computadoras, software de red, medios y dispositivos de interconexión forman un sistema de comunicación
 - Ejemplos: red de sala de PCs, red Universitaria, Internet

Componentes de un sistema de comunicación

- Fuente (Software).
- Emisor/Transmisor (Hardware).
- Medio de transmisión y dispositivos intermedios (Hardware).
- Procesos intermedios que tratan la información (Software y Hardware).
- Receptor (Hardware).
- Destino (Software).
- Otros: Protocolos (Software), Información, mensaje transmitido (Software).
- Señal de Información, materialización del mensaje sobre el medio (Hardware?).

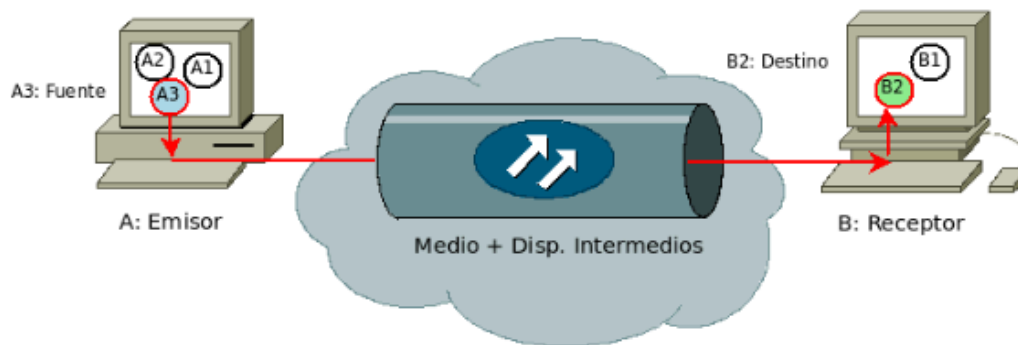


Figure: Sistema de Comunicaciones.

Componentes de una red

- Fuera del punto de vista sistémico podemos ver un gran número de componentes:
 - Computadoras, en el modelo de Internet: Hosts (PCs, laptops, servidores)
 - Routers/switches, Gateways, AP (Access points)
 - NIC (placas de red), Modemos.
 - Vínculos/enlaces: conformados por:
 - Medios: cables, fibras ópticas, señales electromagnéticas, antenas, interfaces, etc.
 - Programas: Browsers, Servidores Web, Clientes de Mail, Servidores de Streaming
 - Etc
- Las componentes de la red deben interactuar y combinarse a través de reglas.

Protocolos

Protocolos

Protocolo

Es el conjunto de conductas y normas a conocer, respetar y cumplir no sólo en el medio oficial ya establecido, sino también el

social y laboral.

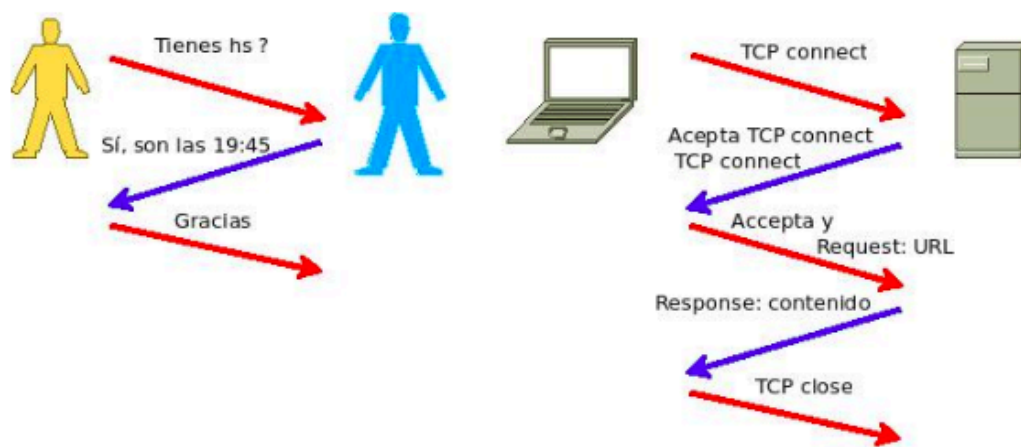


Figure: Protocolo

Nota: La imagen compara la comunicación humana con la de los dispositivos, mostrando que ambos necesitan reglas y secuencias ordenadas (protocolos) para entenderse.

Un protocolo define el formato, orden de los mensajes intercambiados y las acciones que se llevan a cabo en la transmisión y/o recepción de un mensaje u otro evento.

Protocolo de Red

Conjunto de reglas que especifican el intercambio de datos u órdenes durante la comunicación entre las entidades que forman parte de una red. Permiten la comunicación y están implementados en las componentes.

Protocolos de Redes propietarios

- A principios de los 80', las compañías comenzaron a implementar redes propias (privadas y cerradas).
- Primeras Redes Propietarias.
- Consecuencia: Cada red tenía sus especificaciones propias (protocolos).

- Resultados: Incompatibilidad. La comunicación entre redes era muy difícil, evolución más lenta, carencia de estándares.
- Complejidad de modelos.

Combinación de protocolos

- La cantidad de componentes de red a interactuar genera complejidad, se requiere una organización de las mismas.
- Se requieren Modelos de Organización.
- Modelo en Capas: Layering, divide la complejidad en componentes reusables.
 - Reduce complejidad en componente más pequeñas.
 - Las capas de abajo ocultan la complejidad a las de arriba, abstracción.
 - Las capas de arriba utilizan servicios de las de abajo: Interfaces, similar a APIs.
 - Los cambios en una capa no deberían afectar a las demás si la interfaz se mantiene.
 - Facilita el desarrollo, evolución de las componentes de red asegurando interoperabilidad.
 - Facilita aprendizaje, diseño y administración de las redes

Modelo OSI

Modelo TCP-IP

- Modelo que se convirtió en estándar.
- ¿Qué protocolos se encuentran en Internet?
 - Modelo Abierto.
 - Varios protocolos de nivel de enlace: Ethernet, t, 802.3, PPP, HDLC, Frame-Relay, 802.11a/b/g/n/ac, MPLS, DOCSIS, GPON, etc (No definidos por TCP/IP).
 - Protocolos propios de Internet y Transporte (Núcleo): ARP, IP, ICMP, TCP, UDP, OSPF, BGP, etc.
 - Protocolos de Aplicaciones: DNS, HTTP, FTP, SSH, SMTP, IMAP, etc.

- API abierta para generar nuevos protocolos.
- Modelo de 5 (cinco) capas:
 - Capa de aplicación(Process/Application)
 - Capa de transporte o Host-to-host.
 - Capa de Internet o Internetworking.
 - Capa de Enlace(Link Layer).
 - Capa Física
- Por simplicidad algunos autores hablan de 4 capas, agrupando la capa de enlace y la capa física en una sola capa que llaman capa de acceso a la Red.

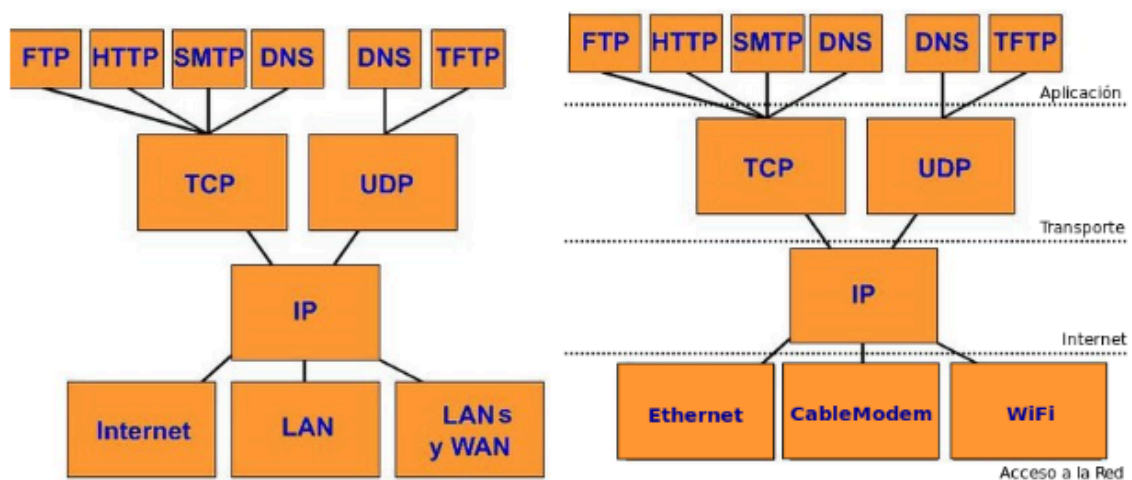


Figure: Modelo TCP/IP

Comparación OSI vs TCP-IP

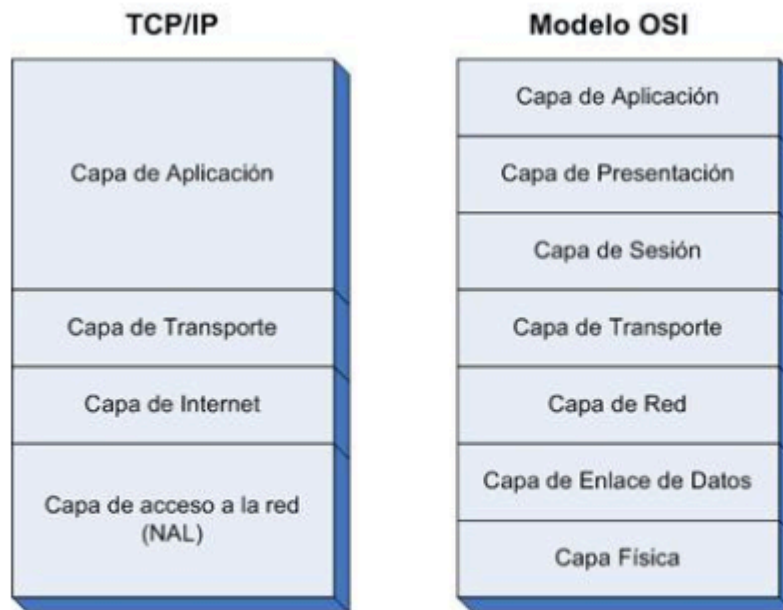


Figure: Modelo OSI vs TCP/IP

- Similitudes:
 - Ambos se dividen en capas.
 - Ambos tienen capas de aplicación, aunque incluyen servicios distintos.
 - Ambos tienen capas de transporte similares.
 - Ambos tienen capa de red similar pero con distinto nombre.
 - Se supone que la tecnología es de conmutación de paquetes (no de conmutación de circuitos).
 - Es importante conocer ambos modelos.
- Diferencias
 - TCP/IP combina las funciones de la capa de presentación y de sesión en la capa de aplicación.
 - TCP/IP combina la capas de enlace de datos y la capa física del modelo OSI en una sola capa.
 - TCP/IP más simple porque tiene menos capas.
 - Los protocolos TCP/IP son los estándares en torno a los cuales se desarrolló Internet, de modo que la credibilidad del modelo
 - TCP/IP se debe en gran parte a sus protocolos.

- El modelo OSI es un modelo “más” de referencia, teórico, aunque hay implementaciones.

Encapsulamiento

- Cada capa define su PDU: Protocol Data Unit

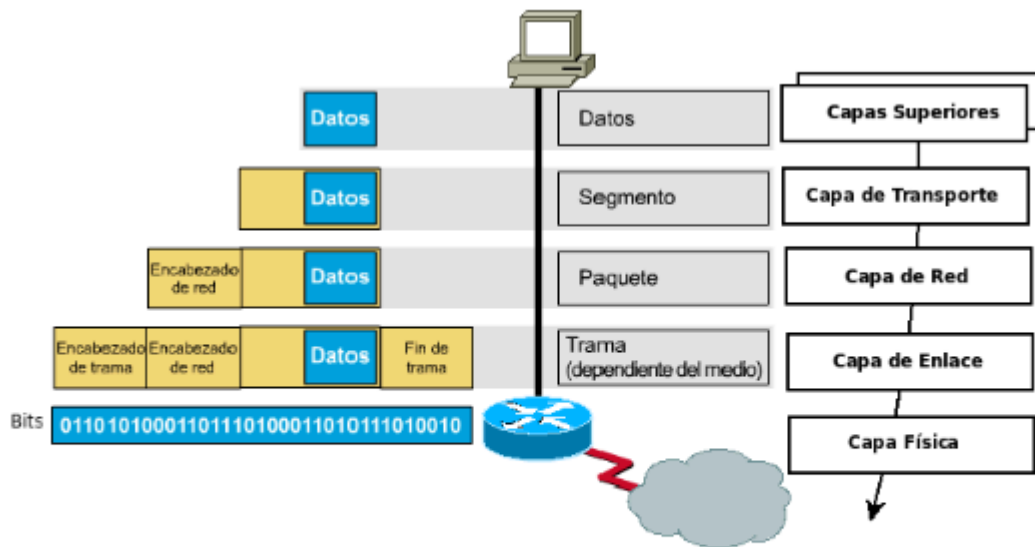


Figure: Encapsulamiento y PDUs

A medida que los datos descienden por las capas, sufren un proceso de encapsulamiento: cada capa envuelve los datos en una PDU más grande agregándole sus propios encabezados de control

Dispositivos y capas

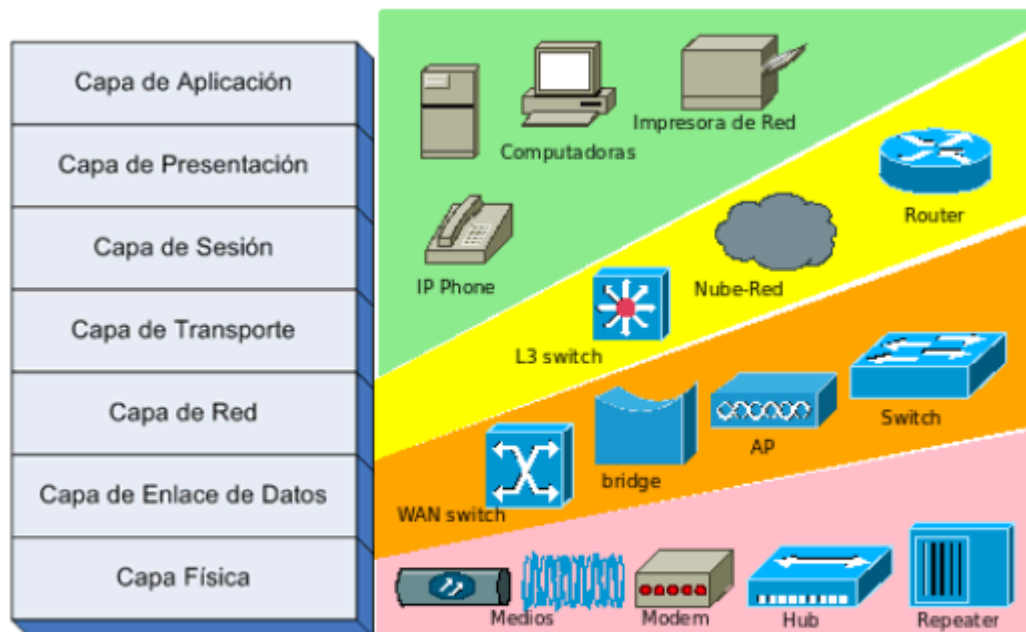


Figure: Dispositivos y Capas

Comunicación entre capas Peer-Peer

- Cada capa usa el servicio de la de abajo.
- Cada capa se comunica con la capa del otro extremo.

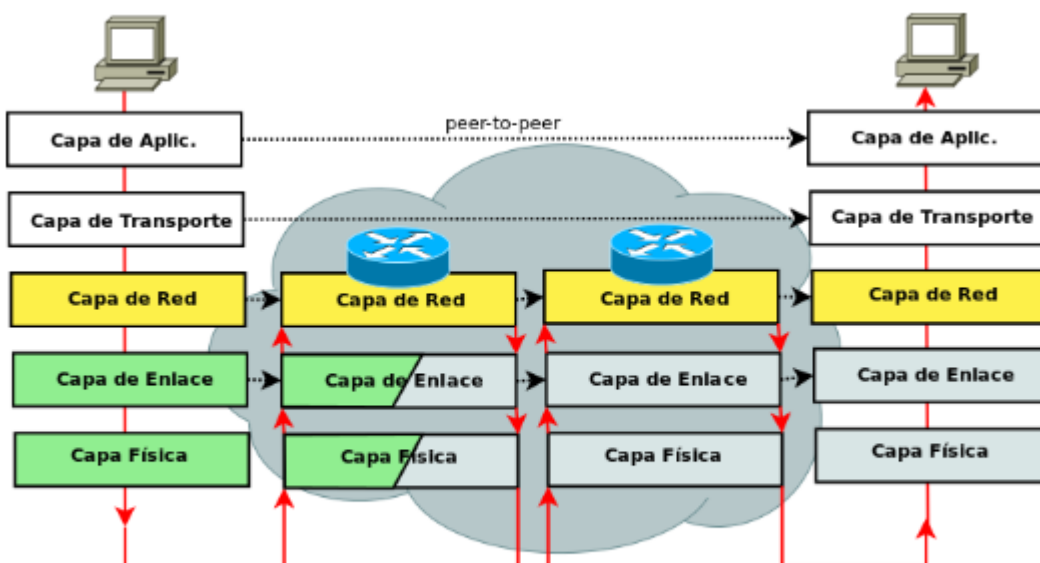


Figure: Comunicación Peer-Peer

Clasificación de Redes

Clasificación de Redes

- Diferentes clasificaciones de acuerdo a diferentes aspectos.
- Se pueden mencionar:
 - Clasificación por cobertura, distancia, alcance.
 - Clasificación por acceso abierto o privado
 - Clasificación por topología física
 - Clasificación por tipo de conexión/medio
 - Etc

Clasificación por cobertura

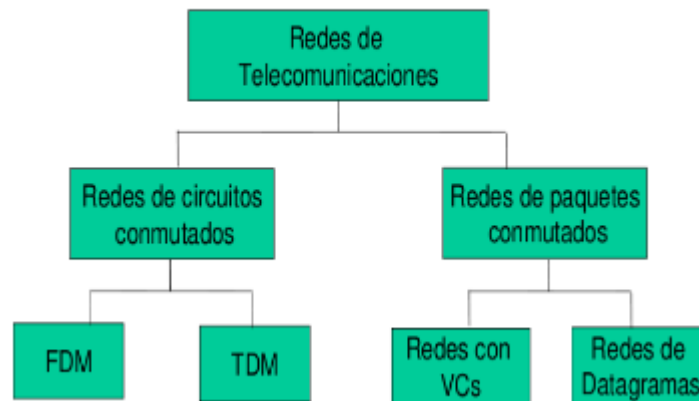
- LAN: (Local Area Network). Red de cobertura local. Ethernet, Wi-Fi.
- MAN: (Metropolitan Area Network). red de cobertura metropolitana, dentro de una ciudad. MetroEthernet, MPLS, Wi-Max.
- WAN: (Wide Area Network). red de cobertura de área amplia. Geográficamente distribuida. PPP, Frame-Relay, MPLS, HDLC, SONET/SDH.
- SAN: (Storage Area Network). red de almacenamiento. iSCSI, Fibre Channel, ESCON.
- PAN: red de cobertura personal. Red con alcance de escasos metros para conectar dispositivos cercanos a un individuo. Bluetooth, IrDA, USB.
- Otros términos: CAN (Controller Area Network o Campus Area Network), NAN (Near-me AN, NFC), ...

Clasificación Públicas y Privadas

- **Internet:** red pública global, tecnología TCP/IP.
- **Intranet:** red privada que utiliza la tecnología de Internet.
- **Extranet:** red privada virtualizada sobre enlaces WAN: Internet. Intranet con acceso de usuarios remotos. VPN (Virtual Private Network) IPSec, PPTP, SSL, OpenVPN, L2TP. Una intranet mapeada sobre una red pública como Internet

Clasificación Física de redes

- Redes de Conmutación de Circuitos.
- Redes de Conmutación de Tramas/Paquetes.
 - Servicios Orientados a Conexión. Circuitos Virtuales.
 - Servicios NO Orientados a Conexión. Datagramas.



Internet

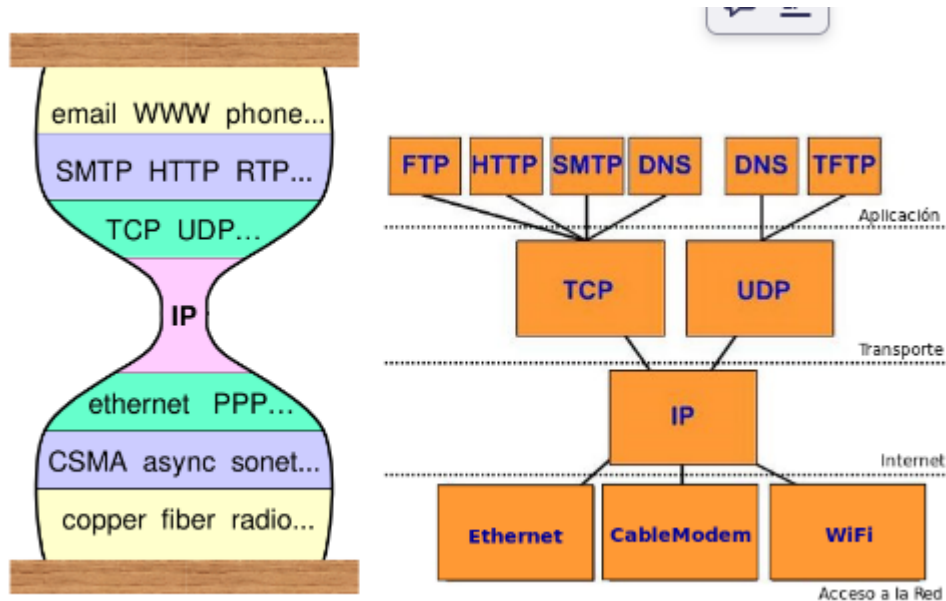
Internet

¿Qué es Internet?

- Es una red de redes de computadoras, descentralizada, pública, que ejecutan el conjunto abierto de protocolos (suite) TCP/IP. Integra diferentes protocolos de un nivel más bajo:
 - Interetworking

Modelo de Internet

Modelo de forma de reloj de arena (hourglass):



Modelo simplificado de Internet

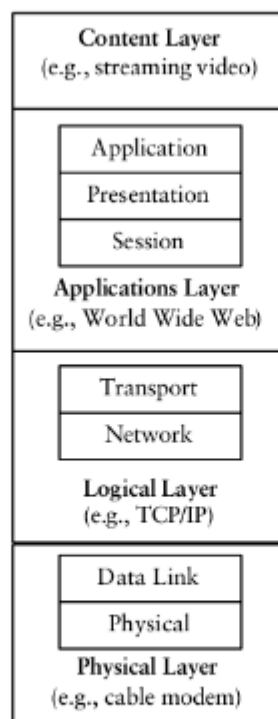


Figure: Modelo Simplificado de Internet de 4 capas

Objetivos/Historia

- Inicios de 1960': Red militar para la guerra fría ?? (aún no existía TCP/IP). Packet Switching Theory: paper de Kleinrock, usar

paquetes en lugar de circuitos en ARPANET: RAND Corp, Leonard Kleinrock del MIT trabajan sobre la red, BBN implementa IMPs.

- Primera vez On-Line 1969: Conectaba las Universidades: Stanford (SRI), Utah, UCLA, UCSB (UC Santa Barbara).
- Nuevo protocolo LAN 1973: Ethernet, Bob Metcalfe en Xerox PARC.
- Cambio a TCP/IP 1983: Desde NCP a TCP/IP. Vinton Cerf y Robert E. Kahn.
- Luego, NSFNET 1985: Red Científica e Investigación, Usada por las Universidades.
- Continuando 1988: Comienza como negocio, nuevas oportunidades.
- Hoy 2020: Tele-trabajo, Clases Virtuales, Redes Sociales,

Request For comments

Son los documentos técnicos oficiales, elaborados por la comunidad de ingenieros de Internet (la IETF), que describen cómo deben funcionar los protocolos (IP, TCP, HTTP, etc.).

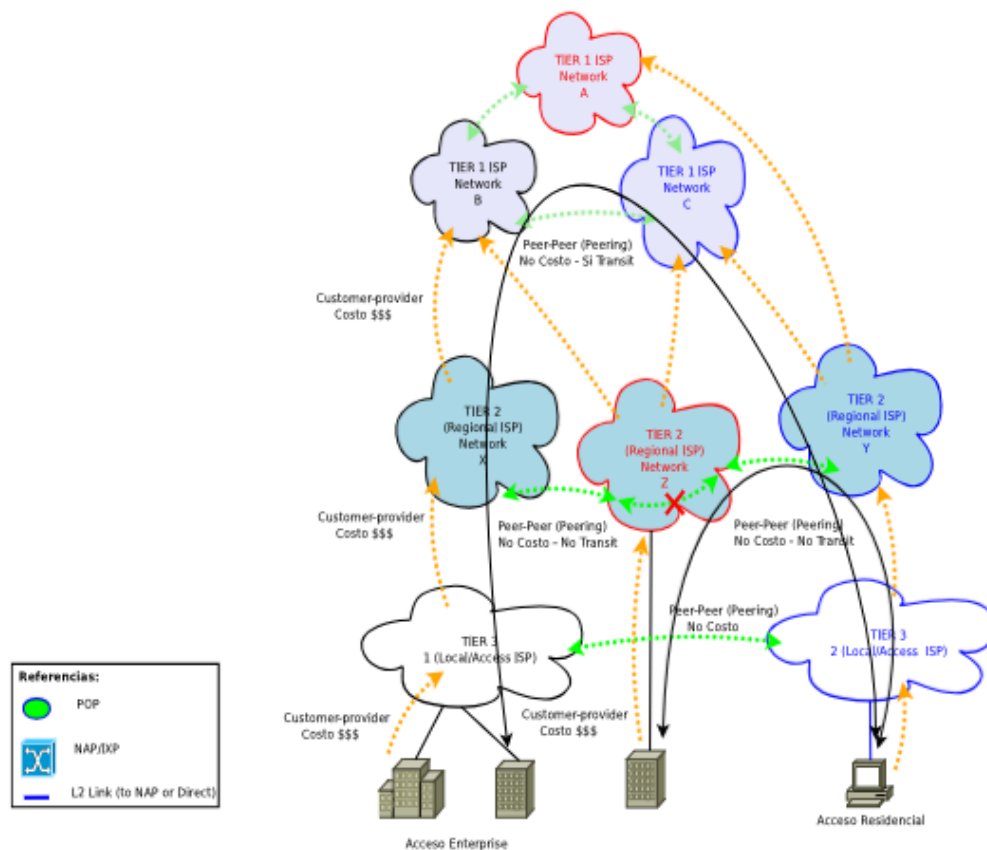
- Categoría STANDARD TRACK. RFC maturity levels.
Proposed Standard: no se requiere implementaciones. Se asigna RFCnnnn.
Internet Standard (STD): existen implementaciones y significativa experiencia operacional. Se retiene el RFCnnnn y se agrega STDxxxx
- Otras Categorías: "Off-track".
INFORMAL/EXPERIMENTAL: otro proceso, se publica como Internet Draft, pero se coloca en otra Cat.
BCP (Best Current Practices): otro proceso. HISTORIC (STD obsoletas): las RFCs se van actualizando o se pueden declarar obsoletas por otras.
FYI: (For Your Information): como INFORMATIONAL

Estructura de Internet

- Estructura en jerarquía, en Tiers.
- Capa de acceso(Edge): Acceso Residenciales, Acceso de Organizaciones.
- Capa de núcleo (Core): dividida en diferentes niveles.
 - Proveedores Regionales
 - Proveedores Nacionales
 - Proveedores Internacionales
 - Proveedores Internacionales en el Tier 1

Core de la estructura

- Tecnologías de fibra óptica, Cobre, Satélites
- Se interconectan mediante POPs (Point Of Presence) con Proveedores.
- Entre proveedores se interconectan mediante NAPs (Network Access Point) o conexiones Directas. Actualmente los NAPs se los llama IXPs (Internet Exchange Point)



Tecnologías de Internet

- Diferentes tecnologías de última milla(acceso) y de redes locales.
- Despliegue local y de acceso de las personas y las organizaciones.
- Ver capa física y enlace modelo OSI.
- Últimos eslabones en la jerarquía

Clase 2

Protocolo HTTP(hasta 1.1) y WEB-Cache

Historia

- WWW (red de sistemas de hipertexto inter-linkeados accesibles vía Internet).
- Desarrollada en 1990 por Tim Berners-Lee en el CERN.
 - Desarrolla el protocolo HTTP y el lenguaje HTML.
- 1993: el primer cliente/browser GUI: Mosaic.
- 1994: Netscape Navigator 1.0.
- Desarrollo de servidores, por ejemplo 1995: Apache Server.
- Buscadores, Indexadores son parte importante del servicio.
- Modelo anterior de interacción: servidor produce, cliente consume.
- Nuevos Modelos: Web 2.0, Tim O'Reilly en 2004, modelo de interacción entre usuarios, consumen y producen mediante servicios especiales: blogs, redes sociales, wikis, multimedia, et

Elementos WEB

- Recurso u Objeto HTTP: ej. web page.
- Referenciado por una URI (Uniform Resource Id): URL (Uniform Resource Location) o URN (Name).
- Formato de URL:
 - protocol://[user:pass@]host:[port]/[path].
- Ejemplo: <http://www.NN.unlp.edu.ar:8080/dir/index.html>.
- URN: no indica ubicación, solo identifican, categorías, poco impl. ej. urn:isbn:0132856204, urn:ietf:rft:2616.
- Objetos pueden ser página web (web page), imágenes JPEG, PNG, GIF, Java Applet, archivos de multimedia: MP3, AVI, etc.

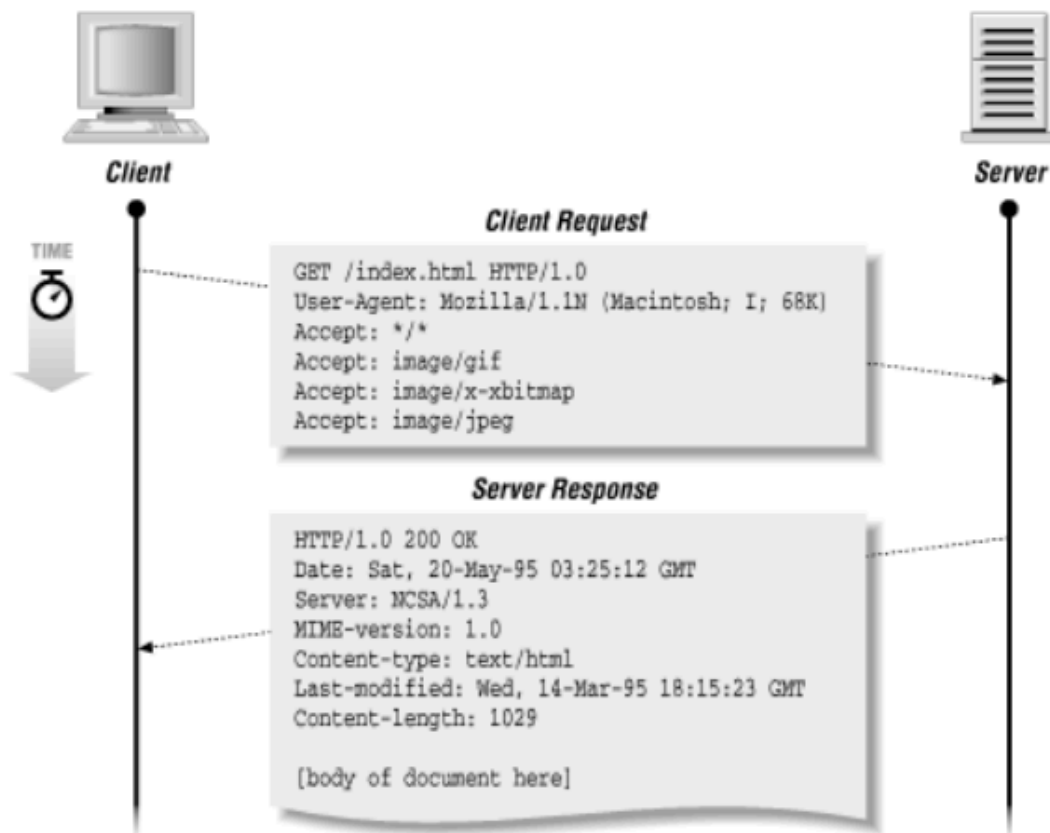
- Páginas web: archivo HTML que incluye v'nculos o directamente otros objetos

Funcionamiento HTTP

Funcionamiento HTTP

- Modelo cliente/servidor, Request/Response (sin estados -stateless)
- Protocolo que corre sobre TCP (requiere protocolo de transporte confiable), usa el puerto 80 por default.
- El cliente escoge cualquier puerto no privilegiado.
- Trabaja sobre texto ASCII, permite enviar información binaria con encabezados MIME.
- Clientes (llamados browsers o navegadores): Firefox, IE, Opera, Safari, Chrome
- Servidores: Apache Server, MS IIS, NGINX , Google GWS, Tomcat

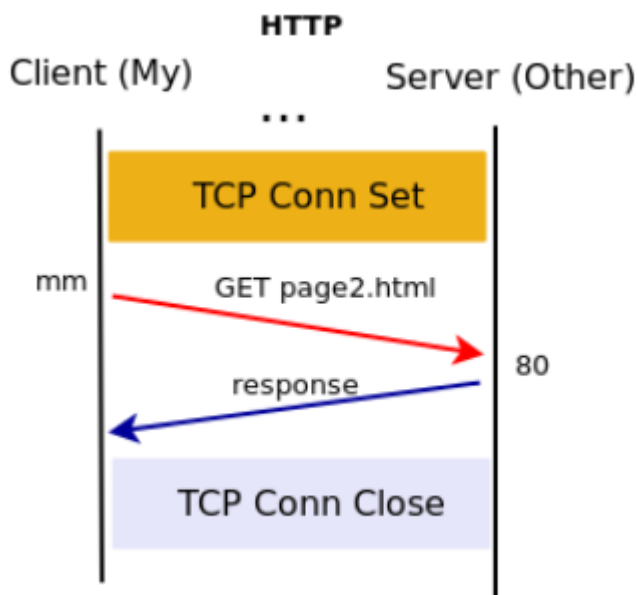
Esquema



Versión HTTP 0.9

- Primera versión de HTTP fue la 0.9 nunca se estandarizó.
- Pasos para obtener un documento:
 1. Establecer la conexión TCP
 2. HTTP Request vía comando GET.
 3. HTTP Response enviando la página requerida.
 4. Cerrar la conexión TCP por parte del servidor.
 5. Si no existe el documento o hay un error directamente se cierra la conexión

Diagrama de pasos HTTP



- Solo una forma de Requerimiento.
- Solo una forma de Respuesta.
- Request/Response sin estado.

Request ::= GET <document-path> <CR><LF>

Response ::= ASCII chars HTML Document.

GET /hello.html <CR><LF>

GET / <CR><LF>

Versión HHTTP 1.0

Versión HTTP 1.0

- Versión de HTTP 1.0 estándar [RFC-1945]
- Define formato, proceso basado en HTTP 0.9:
 - Se debe especificar la versión en el requerimiento del cliente.
 - Para los request, define diferentes métodos HTTP.
 - Define códigos de respuesta.
 - Admite repertorio de caracteres, además del ASCII, como ISO-8859-1, UTF-8, etc.
 - Admite MIME (No solo sirve para descargar HTML e imágenes).
 - Por default NO utiliza conexiones persistentes..

Request

```
<Method> <URI> <Version>  
[<Headers Opcionales>  
<Blank>  
[<Entity Body Opcional>  
<Blank>  
<Method HTTP 1.0> ::= GET, POST, HEAD, PUT,  
DELETE, LINK, UNLINK
```

```
<HTTP Version> <Status Code> <Reason Phrase>  
[<Headers Opcionales>  
<Blank>  
[<Entity Body Opcional>
```

Ejemplos de respuestas

```
HTTP/<version> 200 OK  
HTTP/<version> 301 Moved Permanently  
HTTP/<version> 400 Bad Request  
HTTP/<version> 403 Access Forbidden
```

```
HTTP/<version> 404 Not Found
HTTP/<version> 405 Method Not Allowed
HTTP/<version> 500 Internal Server Error (CGI Error)
HTTP/<version> 501 Method Not Implemented
```

Métodos

- **GET**: obtener el documento requerido. Puede enviar información, pero no demasiada. Es enviada en la URL. Formato ? var1=val1&var2=val2.... Limitación de tamaño de URL por parte de las implementaciones. NO espera recibir datos en body.
- **HEAD**: Idéntico a GET, pero sólo requiere la meta información del documento, por ejemplo, su tamaño. Usado por clientes de caché.
- **POST**: hace un requerimiento de un documento, pero también, envía información en el Body. Generalmente, usado en el fill-in de un formulario HTML(FORM). Puede enviar mucha más información que un GET.
- **PUT**: usado para reemplazar un documento en el servidor. En general, deshabilitada. Utilizado, por ejemplo, por protocolos montados sobre HTTP, como WebDAV [WDV].
- **DELETE**: usado para borrar un documento en el servidor. En general, deshabilitada. También, puede ser utilizada por WebDAV.
- **LINK, UNLINK**: establecen/des-establecen relaciones entre documentos.

GET Response

```
GET /index2.html HTTP/1.0
User-Agent: telnet/andres (GNU/Linux)
Host: estehost.com
Accept: */*

HTTP/1.1 200 OK
Date: Mon, 21 Apr 2008 00:28:51 GMT
Server: Apache/2.2.4 (Ubuntu)
Last-Modified: Mon, 21 Apr 2008 00:18:14 GMT
ETag: "a3b36-1f-91d5d80"
```

```
Accept-Ranges: bytes
Content-Length: 31
Connection: close
Content-Type: text/plain
```

HEAD Response

```
HEAD /index.html HTTP/1.0
User-Agent: telnet/andres (GNU/Linux)
Host: estehost.com
Accept: */*
HTTP/1.1 200 OK
Date: Mon, 21 Apr 2008 00:40:25 GMT
Server: Apache/2.2.4 (Ubuntu)
Last-Modified: Mon, 21 Apr 2008 00:18:14 GMT
ETag: "a3b36-1f-91d5d80"
Accept-Ranges: bytes
Content-Length: 31
Connection: close
Content-Type: text/plain
```

```
<HTML>
<H1> HOLA </H1>
...
</HTML>
```

POST Response

```
POST /index.html HTTP/1.0
User-Agent: telnet/andres (GNU/Linux)
Host: estehost.com
Accept: */*
Content-Type: text/plain
Content-Length: 10
1234567890
HTTP/1.1 200 OK
Date: Mon, 21 Apr 2008 00:37:22 GMT
```

```
Server: Apache/2.2.4 (Ubuntu)
Last-Modified: Mon, 21 Apr 2008 00:18:14 GMT
ETag: "a3b36-1f-91d5d80"
Accept-Ranges: bytes
```

```
Content-Length: 31
Connection: close
Content-Type: text/plain
```

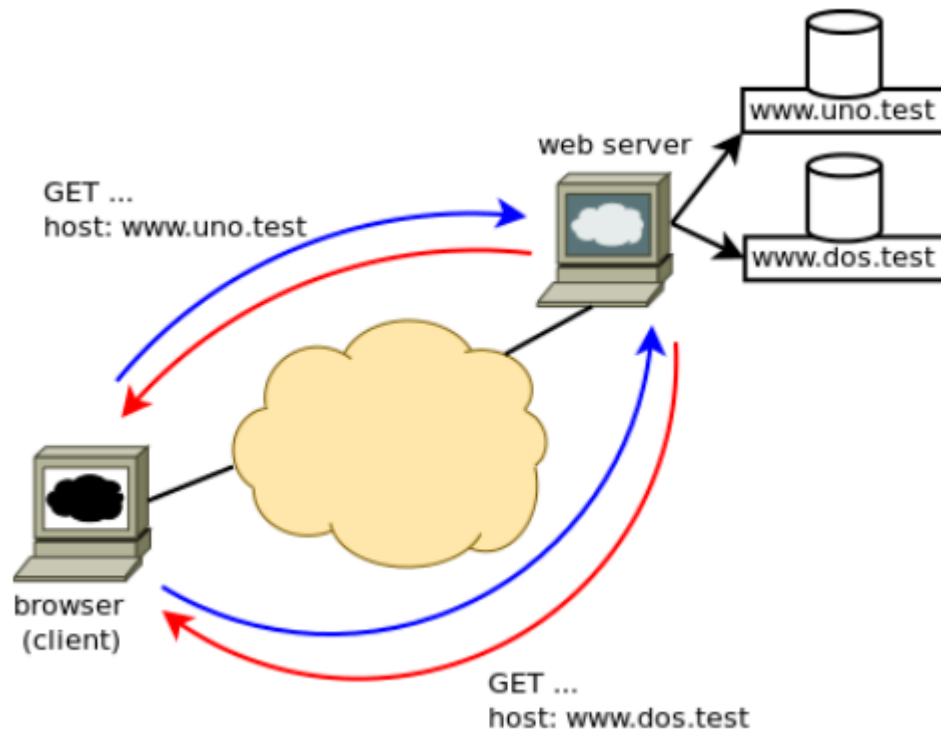
```
<HTML>
<H1> HOLA </H1>
...
</HTML>
```

GET con host virtuales

Mediante el parámetro Host se pueden multiplexar varios servicios sobre un mismo host.

```
? ./mozilla http://www.uno.test
? ./mozilla http://www.dos.test
? host www.uno.test
www.uno.test has address 192.168.0.2
? host www.dos.test
www.dos.test has address 192.168.0.2
```

Hosts Virtuales



Autenticación HTTP

- HTTP/1.0 contempla autenticación con WWW-Authenticate Headers.
- Encabezados, el cliente y el servidor intercambiar información auth.
- El servidor, ante un requerimiento de un documento que requiere autenticación, enviará un mensaje 401 indicando la necesidad de autenticación y un Dominio/Realm.
- El navegador solicitará al usuario los datos de user/password (si es que no los tiene cachedos) y los enviará en texto claro al servidor.
- El servidor dará o no acceso en base a esos valores.
- Para los siguientes requerimientos, el navegador usará los valores que tiene almacenados para el Realm solicitado.

Authorization Required

This server could not verify that you are authorized to access the document requested. Either you supplied the wrong credentials (e.g., bad password), or your browser doesn't understand how to supply the credentials required.

Apache/2.2.9 (Debian) DAV/2 SVN/1.5.1 PHP/5.2.6-1+lenny3 with Suhosin-Patch mod_ssl/2.2.9 OpenSSL/0.9.8g Server at

Port 443

Authentication Required

A username and password are being requested by https://

The site says: "Acceso restringido"

User Name:

Password:

Cancel OK

Conexiones persistentes

- En HTTP/1.0 no se contemplaron las conexiones persistentes por default.
- A partir de HTTP/1.1 [RFC-2068], si.
- En HTTP/1.0 se pueden solicitar de forma explícita.

```
GET /index.html HTTP/1.0
Connection: Keep-Alive
User-Agent: telnet/andres (GNU/Linux)
Host: estehost.com
Accept: */*
HTTP/1.1 200 OK
Date: Tue, 22 Apr 2008 01:31:56 GMT
Server: Apache/2.2.4 (Ubuntu)
Last-Modified: Mon, 21 Apr 2008 00:18:14 GMT
```

```
ETag: "a3b36-1f-91d5d80"
Accept-Ranges: bytes
Content-Length: 31
Keep-Alive: timeout=15, max=10
Connection: Keep-Alive
Content-Type: text/html
<HTML>
<H1> HOLA </H1>
</HTML>
GET /index.html HTTP/1.0
Connection: Keep-Alive
...
Connection closed by foreign host.
```

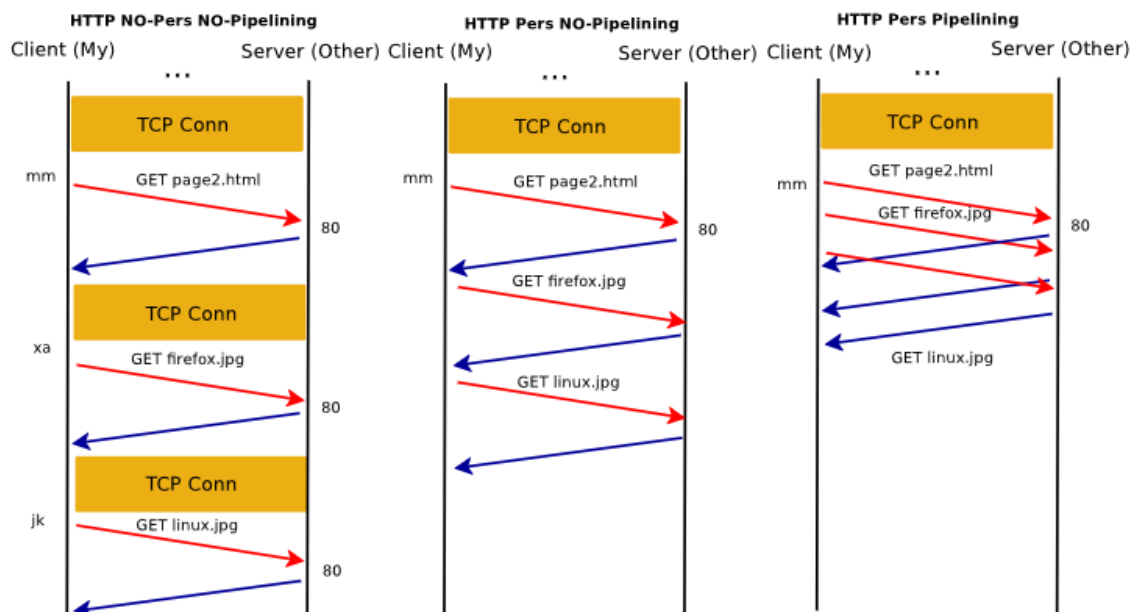
Versión HTTP 1.1

Versión HTTP 1.1

- HTTP/1.1, 1997 con la [RFC-2068] se actualiza con [RFC-2616].
- Nuevos mensajes HTTP 1.1: OPTIONS, TRACE, CONNECT.
- Conexiones persistentes por omisión.
- Pipelining, mejora tiempo de respuestas.

Pipelining

- No necesita esperar la respuesta para pedir otro objeto HTTP.
- Solo se utiliza con conexiones persistentes.
- Mejora los tiempos de respuestas.
- Sobre la misma conexión se debe mantener el orden de los objetos que se devuelven.
- Se pueden utilizar varios threads para cada conexión.
 - Sin pipelining: $1\text{RTT} + \text{FT}$ por cada objeto, n objetos $n\text{RTT} + n\text{FT}$.
 - Con pipelining óptimo: n objetos: $1\text{RTT} + n\text{FT}$.



Trace y connect HHTTP 1.1

- TRACE utilizada para debugging.
- El servidor debe copiar el requerimiento tal cual.
- El cliente puede comparar lo que envía con lo que recibe.
- CONNECT utilizada para generar conexiones a otros servicios montadas sobre HTTP.
- Proxy-Agent genérico

Ejemplo CONNECT

```
CONNECT 127.0.0.1:25
HTTP/1.0 200 Connection Established
Proxy-agent: Apache/2.2.4 (Ubuntu)
220 khartum ESMTP Postfix (Ubuntu)
HELO otrohost.com
250 khartum
QUIT
221 2.0.0 Bye
```

Redirects HTTP

- Redirect temporal 302, indicando la nueva URL/URI
- El user-agent no debería re-direccionarlo salvo que el usuario confirme
- Moved Permanently 301, se indica que cualquier acceso futuro debe realizarse sobre la nueva ubicación (mejora indexadores)
- Se pueden generar problemas con Cookies

Otras Características de HTTP

CGIs Scripts y JavaScript

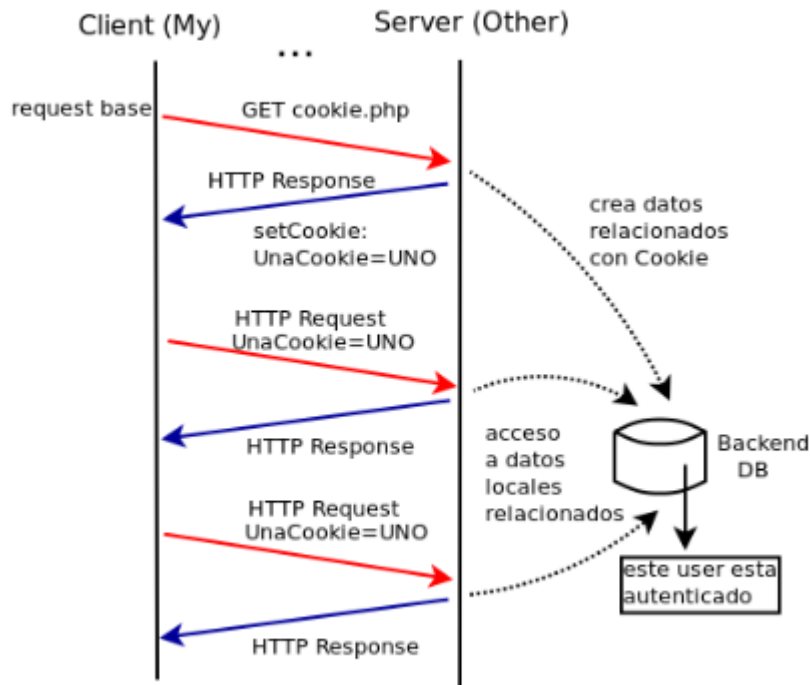
- Necesidad de dinamismo para generar aplicaciones.
- Server-Side Script:
 - Ejecuta del lado del servidor.
 - CGI (Common Gateway Interface): aplicación que interactúa con un servidor web.
 - CGIs leen de STDIN (POST) o de variables de entorno (GET): QUERY STRING datos de usuario.
 - Escriben en la STDOUT response.
 - Deben anteponer el content-type en el header.
 - POST permite enviar más datos.
 - Lenguajes de scripting más flexibles y seguros: PHP, ASP, JSP.
 - Implementados como CGIs, dentro o módulos propios del web-server.

- Client-Side Script
 - Ejecuta del lado del cliente, en el browser
 - JavaScript estándar W3C
 - Usan modelo de objetos DOM (Document Object Model)
 - Otros lenguajes JScript, VBScript.
 - Permiten extensiones como AJAX(Asynchronous JavaScript And XML)
 - AJAX hace requerimientos particulares y no necesita recargar toda la página.
 - Parseo XML para comunicarse.
 - Existen numerosos frameworks que encapsulan esta funcionalidad brindando una interfaz de programación, API fácil de utilizar.
- Server-Side to Server-Side Script:
 - Permiten comunicación entre servidores.
 - Modelo de “objetos” y servicios distribuidos.
 - Conjunto de convenciones para implementar RMI (Remote Method Invocation) sobre HTTP (u otro protocolo de texto).
 - Previo XML-RPC.
 - SOAP (Simple Object Access Protocol).
 - Web-Services.
 - REST.

Cookies

- Mecanismo que permite a las aplicaciones web del servidor “manejar estados”.
- El cliente hace un request.
- El servidor retorna un recurso (un objeto HTTP, como una página HTML) indicando al cliente que almacene determinados valores por un tiempo.
- La Cookie es introducida al cliente mediante el mensaje en el header Set-Cookie: mensaje que indica un par (nombre,valor).

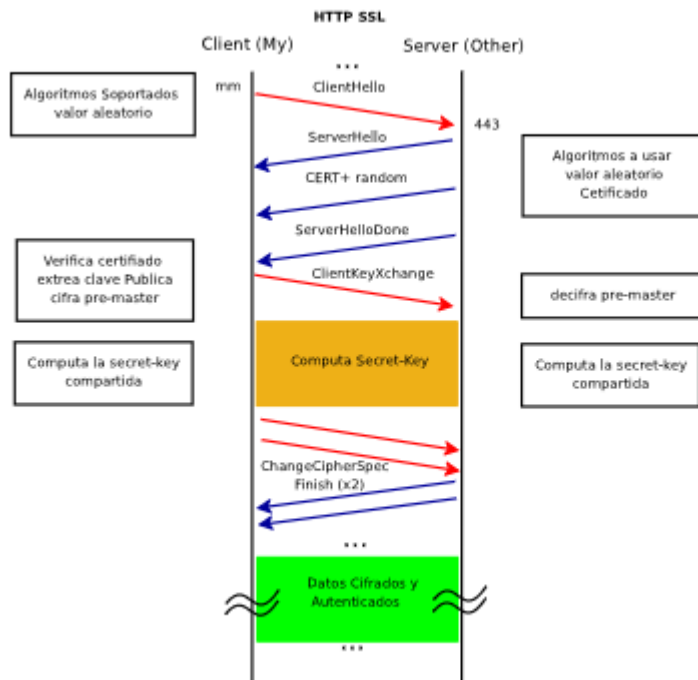
- El cliente en cada requerimiento luego de haber almacenado la Cookie se la enviará al servidor con el header Cookie:.
- El servidor puede utilizarlo o no.
- El servidor puede borrarlo.
- Esta información puede ser usada por client-side scripts.



HTTPS (HTTP sobre TLS/SSL)

- Utiliza el port 443 por default.
- Etapa de negociaci3n previa.

- Luego se cifra y autentica todo el mensaje HTTP (incluso el header)



WEB-Cache

- "Proxiar" y Cachear recursos HTTP
- Objetivos:
 - Mejorar tiempo de respuesta (reducir retardo en descarga).
 - Ahorro en BW (recursos de la red)
 - Balance de carga, atender a todos los clientes
- Se solicita el objeto, si esta en cache y está "fresco" se retorna desde allí (HIT)
- Si el objeto no está o es viejo se solicita al destino y se cachea
- Se puede realizar control de acceso
- Cache del lado del cliente (privada).
- Los web browser tienen sus propias cache locales.
- Los servidores agregan headers:
 - Last-Modified: date
 - ETag: (entity tag) hash
- Requerimientos condicionales desde los clientes:
 - If-Modified-Since: date
 - If-None-Match: hash

- Respuestas de los servidores:
 - 304 Not Modified.
 - 200 OK
- Los cache como servers funcionan como Proxy (shared cache).
- Son servidores a los clientes y clientes a los servidores web.
- Los instalan ISP o redes grandes que desean optimizar el uso de los recursos.
- Existen:
 - Proxy no-transparente.
 - Proxy transparentes.
 - Proxy en jerarquía o mesh (ICP y HTCP).
 - CDN (Content Delivery Network), funcionan por DNS.

